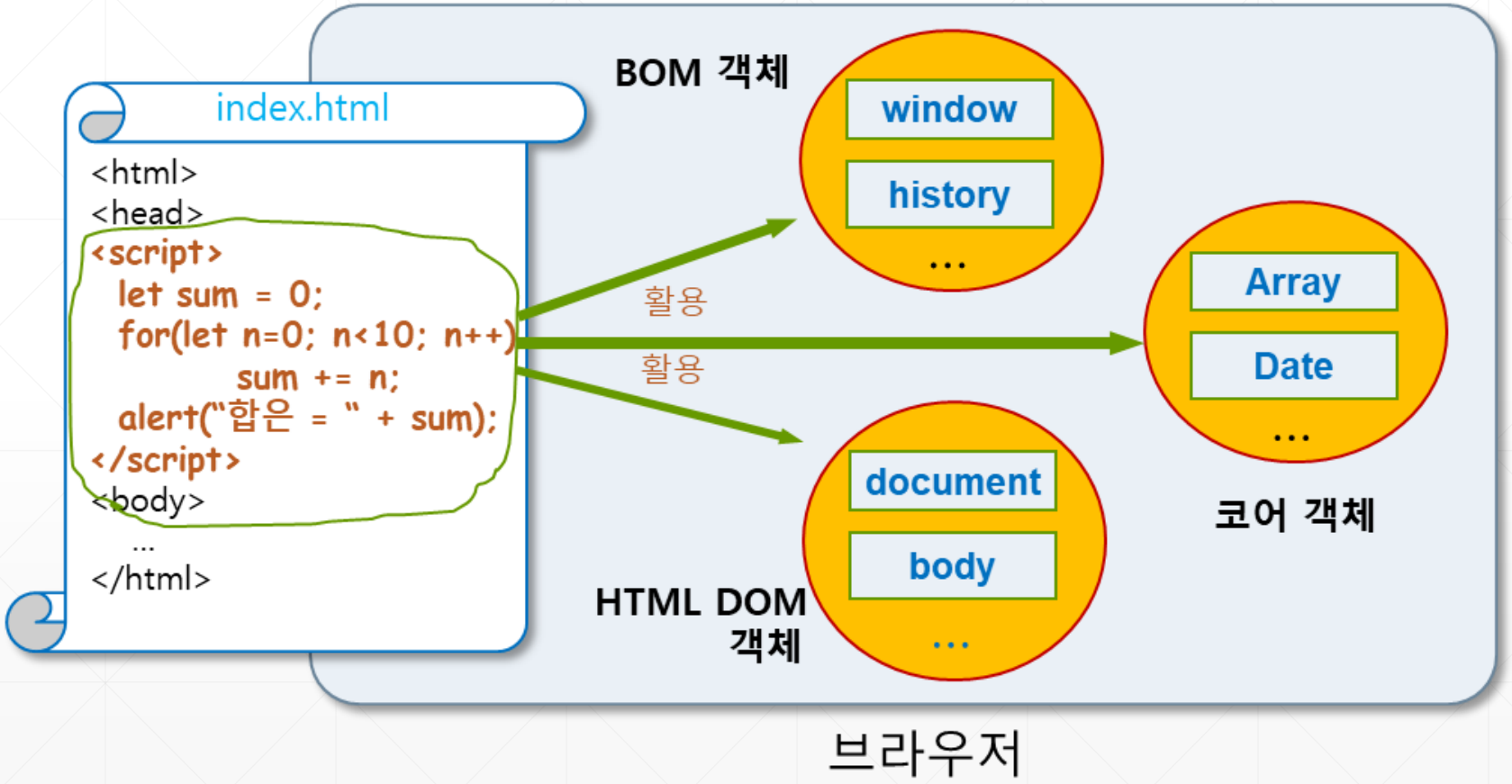
Web Programming

Event Handling

Agenda

- DOM
- Event Handling



DOM

Event Handling

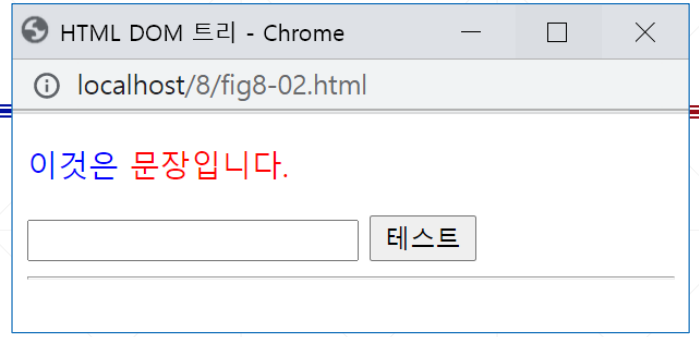
객체 (1/4)

- DOM(Document Object Model)
  - HTML 문서의 내용을 분석하기 위한 구조
  - 자바스크립트는 DOM 조작을 통하여 문서의 내용을 조작 가능

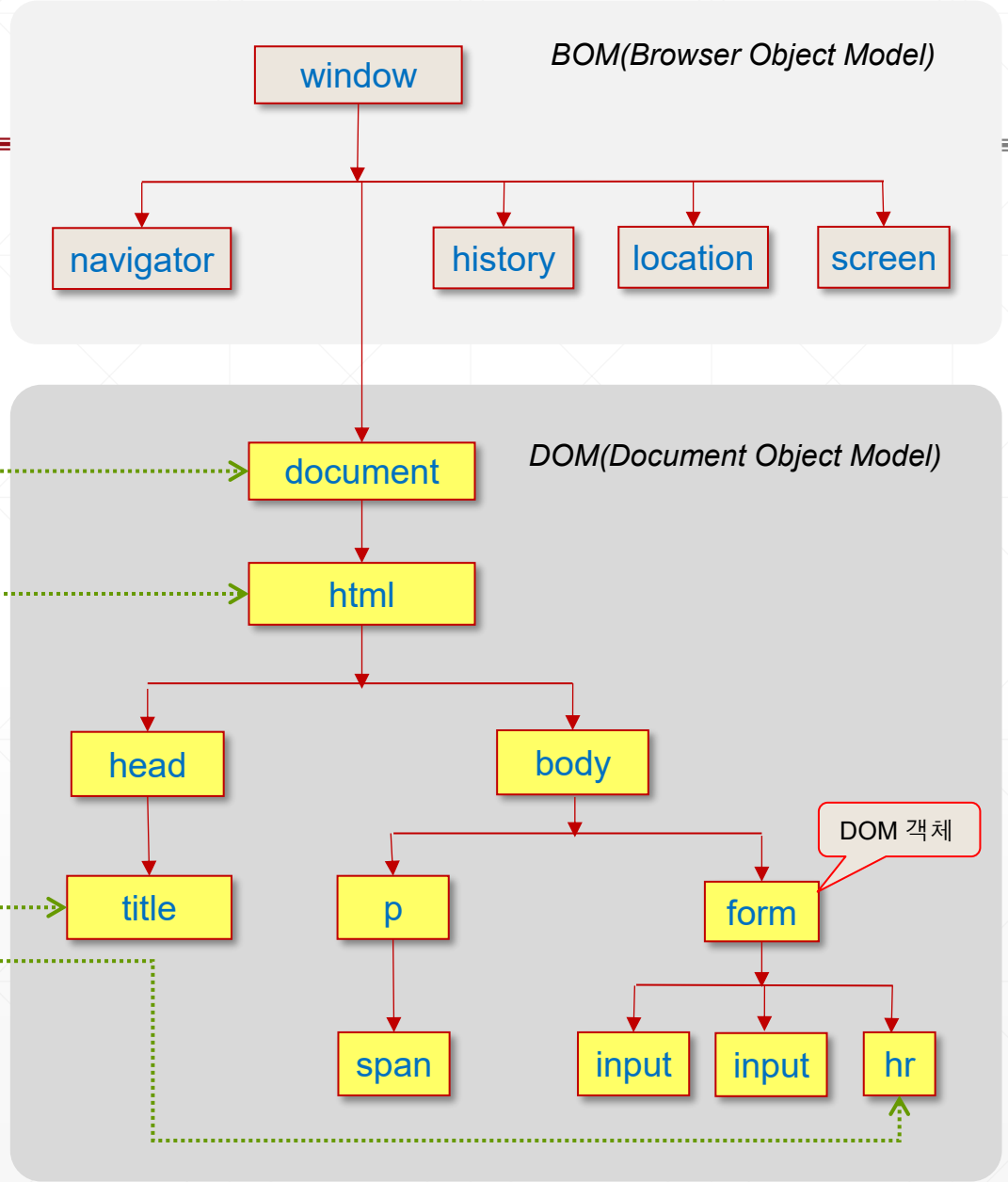
- HTML 문서는 태그 요소의 계층적 구조
  - DOM: 트리 구조의 형태 (root는 document 객체)
    - 트리의 노드: HTML 태그 요소
    - 노드는 속성과 속성값을 가짐
  - HTML 태그 당 DOM 객체가 하나씩 생성, 따라서 DOM 객체의 종류는 HTML 태그 종류만큼!
  - HTML 태그의 포함관계에 따라 DOM 트리에 부모 자식 관계

- DOM 구조는 웹 브라우저에서 확인 가능 (개발자 도구 이용)

DOM (2/4)




```
<!DOCTYPE html>
<html>
<head>
    <title>TML DOM 트리</ </title>
</head>
<body>
<p style="color:blue">이것은
    <span style="color:red">문장입니다.
    </span>
</p>
<form>
    <input type="text">
    <input type="button" value="테스트">
    <hr>
</form>
</body>
</html>
```



DOM (3/4)

- 태그 요소는 DOM의 객체로 표현 됨
  - 태그 요소에 포함된 요소는 객체내에 포함된 하위 객체 형태로 표현
- 태그 속성은 DOM 객체의 속성으로 표현 됨

- 요소 전체가 하나의 객체

<a href= “URI"> My Link </a>

  - href는 속성, ”URI”는 href 속성의 값
  - “My Link”는 <a>요소 (DOM 객체)의 텍스트 노드

DOM (4/4)

- 브라우저가 HTML 태그를 화면에 그리는 과정
  1. 브라우저가 DOM 트리의 틀(document 객체) 생성
  2. 브라우저가 HTML 태그를 읽고 DOM 트리에 DOM 객체 생성
  3. 브라우저는 DOM 객체를 화면에 출력
  4. HTML 문서 로딩이 완료되면 DOM 트리 완성
  5. DOM 객체 변경 시, 브라우저는 해당 HTML 태그의 출력 모양을 바로 갱신

DOM (4/4)

- DOM 조작을 통한 문서 내용/스타일 변경 과정
  1. DOM 트리로부터 원하는 객체 탐색
  2. 해당 DOM 객체에 대한 변경
     - 내용
     - 속성 값
     - 스타일
     - 이벤트 처리
  3. 해당 DOM 객체 기준 객체 추가/삭제 수행

DOM – 객체 탐색 (1/4)

- HTML 문서에서 DOM 객체를 선택하기 위한 메소드

객체 탐색 메소드	결과
document.getElementById(아이디)	DOM 객체
document.getElementsByClassName(클래스)	DOM HTMLCollection
document.getElementsByName(이름)	DOM NodeList
document.getElementsByTagName(태그)	DOM HTMLCollection
document.querySelector(CSS선택자)	DOM 객체
document.querySelectorAll(CSS선택자)	DOM NodeList
document.head	Head 객체
document.body	Body 객체
document.title	Title
…	

DOM – 객체 탐색 (2/4)

- DOM 객체
  - HTML 요소를 표현하는 객체
  - DOM 객체를 바탕으로 내용,속성,스타일,이벤트 처리 등 가능
  - HTML 요소 (HTMLElement)의 속성 + DOM 객체 (Element) 자체 속성/메소드 제공
    - HTMLElement - Web APIs | MDN (mozilla.org)
    - Element - Web APIs | MDN

- HTMLCollection / NodeList
  - DOM 객체들의 집합 (배열과 유사)
  - Length 속성을 통해 길이 획득 가능
  - 인덱싱([])을 통하여 요소 순회 가능
  - HTMLCollection - Web APIs | MDN, NodeList - Web APIs | MDN

DOM – 객체 탐색 (3/4)

- 예제)

```
<!DOCTYPE html>
<html>
<head>
    <title>DOM Test</title>
</head>
<body>
    <div id="intro">
        <a href="http://www.seoultech.ac.kr">My link</a>
        <h1 class="header">My header</h1>
        <ul>
            <li>1</li>
            <li>2</li>
            <li>3</li>
        </ul>
    </div>
</body>
</html>
```

DOM – 객체 탐색 (4/4)

- ID에 의한 객체 탐색

```
var obj = document.getElementById('intro');
console.log(obj.tagName);
console.log(obj.childElementCount);
```

- Tag에 의한 객체 탐색

```
var obj = document.getElementsByTagName('li');
for (node of obj){
    console.log(node.innerHTML);
}
```

- CSS selector에 의한 객체 탐색

```
var obj = document.querySelectorAll('#intro li');
for (node of obj){
    console.log(node.innerHTML);
}
```

DOM – 객체 수정 (1/2)

- 객체의 내용물 수정
  - obj.outerHTML: 객체를 포함하는 태그 요소
  - obj.innerHTML: 객체에 포함되어 있는 태그 요소
  - obj.textContent: 객체에 포함되어 있는 텍스트 요소

- 객체의 속성 수정
  - obj.속성명: 해당 객체의 속성값 조회/설정 가능

- 객체의 스타일 수정
  - obj.style.스타일이름: 해당 객체의 스타일 설정 가능
  - 하이픈('-')이 들어가는 CSS 스타일 이름은 camelCase로 변경

DOM – 객체 수정 (2/2)

- 예제)

```
// a Tag의 내용물 수정
var link = document.getElementsByTagName('a');
link[0].innerHTML="<b>Updated Link</b>";
console.log(link[0].outerHTML);
console.log(link[0].innerHTML);
console.log(link[0].textContent);

// a Tag의 속성 수정
console.log(link[0].href);
link[0].href="http://eclass.seoultech.ac.kr";
link[0].target="_blank";

// a Tag의 스타일 수정
link[0].style.color="red";
link[0].style.fontSize="3em";
```

DOM – 객체 추가/삭제

- 객체의 생성/추가/삭제 가능

객체 생성/추가/삭제 메소드	설명
obj.createElement(*tagname*)	HTML element 생성
obj.removeChild(*element*), obj.remove()	HTML element 삭제
obj.appendChild(*element*)	HTML element 추가
obj.insertBefore(*new, existing*)	Existing 노드 앞에 new 노드 삽입

```
var div = document.getElementById('intro');
var obj = document.createElement('h1');
obj.innerHTML="GoodBye";                          // 객체 생성 및 추가
div.appendChild(obj);

var link = document.getElementsByTagName('a')[0];
div.removeChild(link);                            // 객체 삭제

document.body.insertBefore(obj,div);              // 객체 삽입
```

DOM

Event Handling

DOM Event (1/4)

- 이벤트
  - 마우스 클릭, 키보드 입력, 이미지나 HTML 문서의 로딩, 타이머의 타임아웃 등 사용자의 입력 행위나 문서나 브라우저의 상태 변화를 자바스크립트 코드에게 알리는 통지(notification)
- 이벤트 리스너
  - 발생한 이벤트에 대처하기 위해 자바스크립트로 작성된 로직
- 이벤트 종류
  - HTML5에서 이벤트 종류는 70여가지
  - 이벤트 리스너 이름은 이벤트 이름 앞에 on을 덧붙임
  - 예) onmousedown : mousedown 이벤트의 리스너

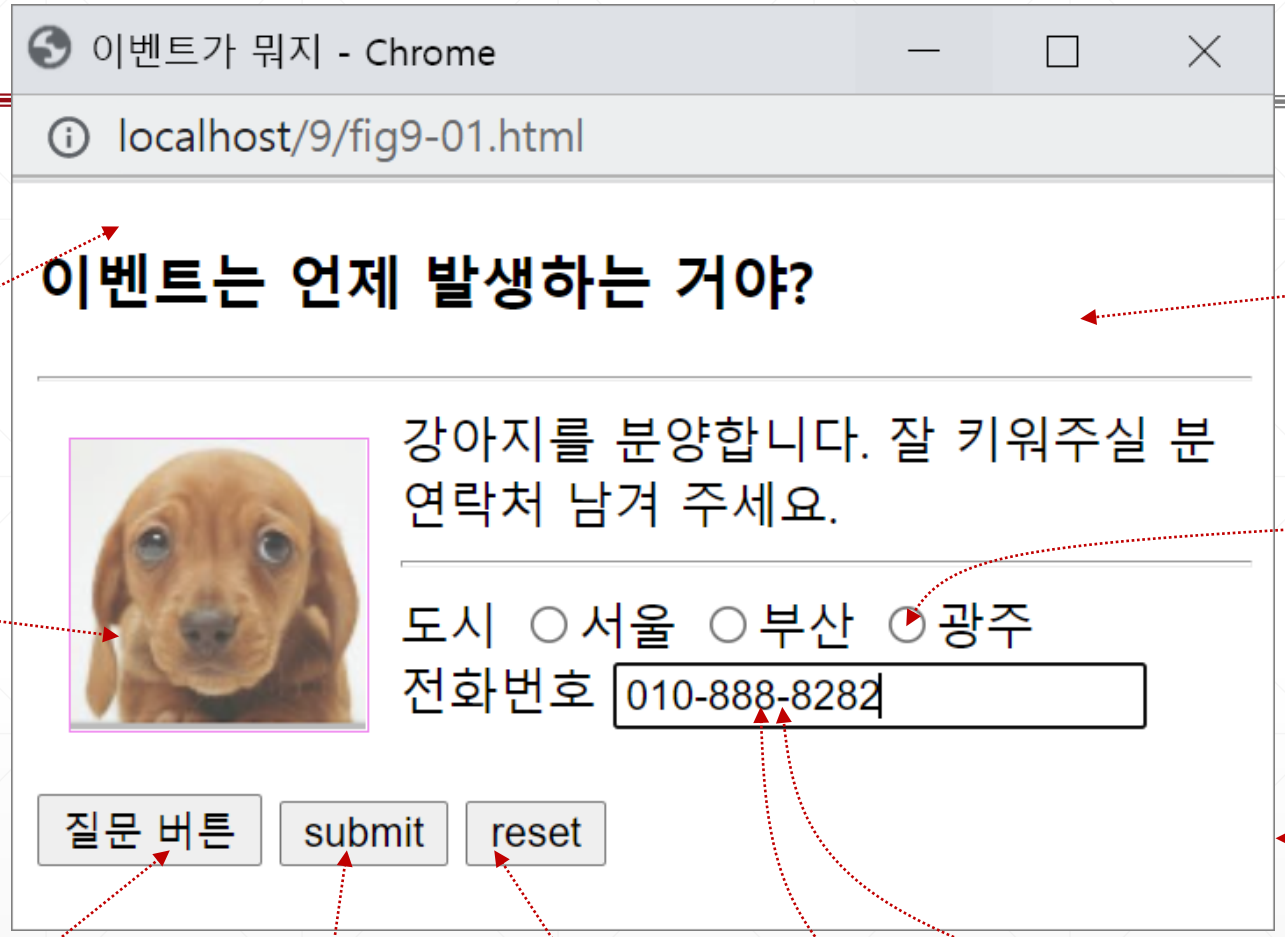
    onkeydown : keydown 이벤트의 리스너

DOM Event (2/4)

- HTML 요소에는 다양한 이벤트들이 발생
  - Click: 클릭했을 때 발생
  - Focus: 포커스를 획득하였을 때 발생
  - Change: 내용이 변경되고 포커스를 상실하였을 때 발생
  - Keydown: 키를 눌렀을 때 발생
  - Load: 로딩이 완료되었을 때 발생
  - Mouseover: 마우스가 객체 위에 올라왔을 때 발생
  - Submit: 폼이 전송될 때 발생
  - …

- Event reference | MDN (mozilla.org) 참고

DOM Event (3/4)



load 이벤트 (HTML 문서 전체 로딩 완료 시)

dblclick 이벤트 (마우스 더블클릭 시)

load 이벤트 (이미지의 로딩 완료 시)

change 이벤트 (라디오버튼 선택 시)

resize 이벤트 (윈도우 크기 변경 시)

click 이벤트 (마우스 클릭 시)

submit 이벤트 (submit 버튼 클릭 시)

reset 이벤트 (reset 버튼 클릭 시)

keydown 이벤트 (키를 누를 때)

keyup 이벤트 (누른 키를 놓을 때)

DOM Event (4/4)

- 예제) My link 링크 클릭시 “Goodbye” H1 요소 추가

```
<body>
    <div id="intro">
        <input name="title" type="text"><br>
        <a>My link</a>
        <h1 class="header">My header</h1>
        <ul>
            <li>1</li>
            <li>2</li>
            <li>3</li>
        </ul>
    </div>

</body>
```

Click Event?

어떻게 연결하는가?

```
<script>
 function addObj() {
   var obj = document.createElement("h1");
   obj.innerHTML = "GoodBye";
   document.body.appendChild(obj);
   console.log("H1 element added");
 }
</script>
```

DOM Event Listener (1/8)

- HTML 요소의 이벤트 발생 시 처리 동작 (이벤트 리스너) 등록 방법
  - HTML 요소의 on[이벤트명] 속성 이용 [권장되지 않음]

```
<a onclick="addObj()">My link</a>

<a onclick="addObj()" onmouseover="this.style.color='red'"
onmouseleave="this.style.color='black'"> My link </a>
```

  - DOM 객체의 이벤트 리스너 속성 이용 (obj.on[이벤트명])

```
document.getElementsByTagName('a')[0].onclick=addObj;
```

  - DOM 객체의 addEventListener() 메소드 이용 (obj.addEventListener('이벤트명'))

```
document.getElementsByTagName('a')[0].addEventListener('click',addObj);
```

DOM Event Listener (2/8)

- 주의! DOM 조작 스크립트의 순서
  - 아래 코드들을 body 상단 <script>에 위치시키면 동작하는가?

```
document.getElementsByTagName('a')[0].onclick=addObj;
또는
document.getElementsByTagName('a')[0].addEventListener('click',addObj);
```

  - 이 코드를 수행할 시점에는 HTML DOM Tree가 구성되기 전!
  - HTML DOM Tree 구성시점과 DOM 조작시점을 맞춰줘야 함

- 방법 1: body의 최하단부에 위치
- 방법 2: 모든 리소스가 로딩된 후 실행

DOM Event Listener (3/8)

- 방법 1: body의 최하단에 위치

```
<body>
  <div id="intro">
    <input name="title" type="text" /><br />
    <a>My link</a>
    <h1 class="header">My header</h1>
    <ul>
      <li>1</li>
      <li>2</li>
      <li>3</li>
    </ul>
  </div>
  <script>
    document.getElementsByTagName('a')[0].addEventListener('click',addObj);
  </script>
```

➢ 모든 body요소가 로딩되어 DOM트리 구성 후 자바스크립트 실행!

DOM Event Listener (4/8)

- 방법 2: 모든 리소스가 로딩된 후 실행

```
<head>
  <script>
    function addObj() {
      var obj = document.createElement("h1");
      obj.innerHTML = "GoodBye";
      document.body.appendChild(obj);
      console.log("H1 element added");
    }

    window.onload = function () {
      document.getElementsByTagName("a")[0].onclick = addObj;
    };
  </script>
</head>
```

- window.onload = 모든 리소스가 로딩되면 발생하는 이벤트

DOM Event Listener (5/8)

- Event 객체
  - Event 핸들러에 자동으로 전달되는 객체
  - Event 관련 부가정보와 부가 기능들을 제공
  - event, e, evt 등으로 명명하여 사용

- 예) KeyboardEvent KeyboardEvent - Web APIs | MDN (mozilla.org)
  - 키보드 관련 이벤트 객체
  - KeyboardEvent.key: 이벤트가 나타내는 키의 키 값을 나타내는 문자열을 반환
  - KeyboardEvent.ctrlKey: 키 이벤트가 생성될 때 Ctrl 키가 활성화된 경우, true 값 반환

- Event - Web APIs | MDN (mozilla.org) 참고

DOM Event Listener (6/8)

Event 객체에 들어있는 정보

- 현재 발생한 이벤트에 관한 다양한 정보
  - 이벤트 객체의 프로퍼티와 메소드로 알 수 있음
- 이벤트의 종류마다 조금씩 다름
  - 이벤트 객체의 공통 멤버

멤버	종류	설명
type	프로퍼티	현재 발생한 이벤트의 종류를 나타내는 문자열(click, load 등)
target	프로퍼티	이벤트를 발생시킨 객체(DOM 객체 혹은 HTML 태그)
currentTarget	프로퍼티	현재 이벤트 리스너를 실행하고 있는 DOM 객체
defaultPrevented	프로퍼티	이벤트의 디폴트 행동이 취소되었는지를 나타내는 true/false 값
preventDefault()	메소드	이벤트의 디폴트 행동을 취소시키는 메소드

DOM Event Listener (7/8)

- 예제) 텍스트 입력 시 해당 내용을 “My Header” H1요소의 내용으로 교체

1. 텍스트 입력창의 내용을 H1요소로 교체하는 함수 생성

```
let setTitle = (event) => {
        document.getElementsByClassName("header")[0].innerHTML = event.target.value;
};
```

2. 텍스트 객체 선택

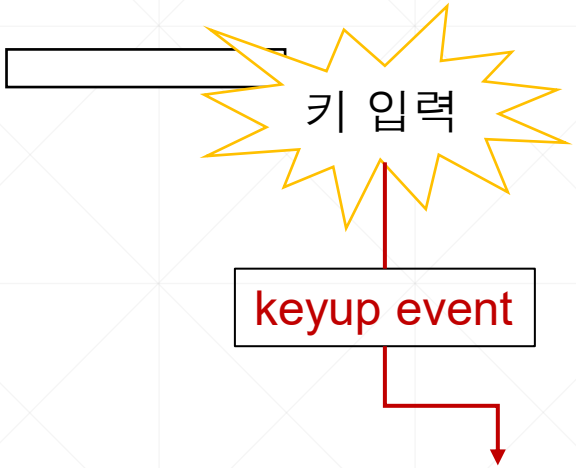
```
document.getElementsByName('title')[0]
```

3. 텍스트에 키보드 입력 이벤트 처리 함수 (콜백) 등록

```
document.getElementsByName('title')[0].addEventListener('keyup',setTitle);
```

DOM Event Listener (8/8)

■ 실행 흐름



keyup event
→ KeyboardEvent {
key, altKey, ctrlKey, …
}
→ Event {
target, timestamp, …
} //해당 이벤트가 발생한 element

```
function setTitle(e) {
    document.getElementsByClassName('header')[0].innerHTML=e.target.value;
};
```